

I. REPRÉSENTATION BINAIRE D'UN NOMBRE RÉEL

1. Écriture de position

Comme la notation décimale, la notation binaire permet aussi de représenter les nombres à virgule.

En notation décimale, les chiffres de gauche représentent les unités, les dizaines et ainsi de suite et ceux à droite de la virgule, les dixièmes, les centièmes, etc ...

$$\begin{array}{ccccccc}
 & 1 & 2 & 3 & 4 & , & 5 & 6 & 7 \\
 \text{un millier} & \text{2 centaines} & \text{3 dizaines} & \text{4 unités} & \text{5 dixièmes} & \text{6 centièmes} & \text{7 millièmes} \\
 1 \times 10^3 & + 2 \times 10^2 & + 3 \times 10^1 & + 4 \times 10^0 & + 5 \times 10^{-1} & + 6 \times 10^{-2} & + 7 \times 10^{-3}
 \end{array}$$

En notation binaire, les chiffres de droites représentent des demis, des quarts, des huitièmes, etc ...

$$\begin{array}{ccccccc}
 & 1 & 1 & 0 & 1 & , & 0 & 1 & 1 \\
 \text{"1" huit} & \text{"1" quart} & \text{"0" deux} & \text{"1" unité} & \text{"0" demi} & \text{"1" quart} & \text{"1" huitième} \\
 1 \times 2^3 & 1 \times 2^2 & 0 \times 2^1 & 1 \times 2^0 & 0 \times 2^{-1} & 1 \times 2^{-2} & 1 \times 2^{-3}
 \end{array}$$

C'est l'écriture binaire de $8 + 4 + 0 + 1 + 0 + 0,25 + 0,125 = 13,375$

2. De l'écriture décimale à la notation binaire et vice-versa

☞ **Première méthode** : on décompose à l'aide des puissances de 2

- $17,625 = 1 \times 16 + 0 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1 + 1 \times 0,5 + 0 \times 0,25 + 1 \times 0,125 = 10001,101$
- $(1,0111)_2 = 1 \times 1 + 0 \times 0,5 + 0 \times 0,25 + 0,0625 = 1,4375$

☞ **Deuxième méthode** : comme un nombre décimal peut s'écrire sous la forme $\frac{A}{10^n}$ avec A un entier relatif et n un entier naturel, un nombre **dyadique** est un nombre qui peut s'écrire sous la forme $\frac{A}{2^n}$. Par exemple, $\frac{14}{8}$ est un nombre dyadique : $\frac{14}{8} = \frac{14}{2^3} = 1,75$.

- Exemple : $\frac{45}{4}$ est un nombre dyadique. Trouvons son écriture binaire :

- ♦ Convertir 45 en binaire : $32 + 8 + 4 + 1 = \dots 101101$
- ♦ Revenir à la définition de l'écriture binaire à l'aide des puissances de 2
 $\dots 2^5 + 2^3 + 2^2 + 2^0$
- ♦ Diviser le résultat par 2^2
- ♦ Ainsi, $\frac{45}{4} = \frac{25}{2}$
- ♦ Par analogie avec les décimaux, on peut écrire que $\frac{45}{4} = \dots$

Avec les décimaux, diviser par 10^n , $n > 0$, revient à décaler la virgule de n rang vers la ...
 De même, diviser par 2^n l'écriture binaire d'un nombre, revient à décaler la virgule de n rang vers la ...

- Convertir de même $\frac{111}{16}$, $\frac{54}{64}$

- ☛ **Troisième méthode** : on écrit la partie entière en base 2 et on obtient les bits de la partie non entière par l'algorithme décrit dans l'exemple qui suit.

Conversion du nombre décimal 0,1

Étape	Partie non entière	(Partie non entière) × 2	Bits successifs de la partie entière du nombre obtenu
1	0,1	0,2	0
2	0,2	0,4	0
3	0,4	0,8	0
4	0,8	1,6	1
5	0,6	1,2	1
6	0,2	0,4	0
7	0,4	0,8	0
8	0,8	1,6	1
9	0,6	1,2	1

Conclusion : 0,1 n'a pas d'écriture finie en binaire (équivalent à $\frac{1}{10}$).

De même, 0,2 = 0,2 non plus.

3. Représentation d'un nombre réel en machine : le type float

En machine, pour représenter un nombre réel, on doit d'abord l'écrire sous forme binaire. Mais la "plupart" des nombres réels ont une **écriture binaire infinie** (même la plupart des nombres décimaux qui ont pourtant une écriture décimale finie). Compte tenu du nombre limité de bits en mémoire, **on ne pourra pas toujours représenter exactement un nombre réel en mémoire.**

En Python une valeur de type float, on dit un flottant, est la représentation d'un nombre réel, très souvent approximative.

En Python, on n'utilise pas le == pour tester l'égalité de 2 nombres de type float.

En pratique, on teste l'égalité de 2 nombres de type float de façon approximative, en vérifiant que la distance entre les 2 est très petite.

Pour tester que deux flottants a et b sont égaux en machine à 10^{-9} près on teste l'inégalité $|a - b| < 10^{-9}$

Exercice : ouvrir Basthon en mode console et tester les lignes suivantes :

```
>>>0.1+0.2==0.3 ..... Résultat : .....
>>>abs(0.1+0.2-0.3)<1e-9 ..... Résultat : .....
```

Conclusion : En machine 0,1+0,2 est différent de 0,3, égal à 0,3 au milliardième près.

Mais, concrètement, comment sont représentés les nombres réels en machine sur n bits ?

L'écriture en virgule flottante est l'analogue en binaire de l'écriture scientifique des nombres décimaux.

Exemple1 :**En décimal :**

$$345,025 = 3,45025 \times 10^2$$

$$0,003569 = 3,569 \times 10^{-3}$$

En binaire :

$$(0,00010111)_2 = (1,0111)_2 \times 2^{-4}$$

$$(1011,0111)_2 = (1,0110111)_2 \times 2^3$$

Dans l'écriture en virgule flottante le chiffre avant la virgule est forcément 1.

En binaire l'écriture en virgule flottante d'un nombre est de la forme : $(-1)^s \times (1,xxx...xxx)_2 \times 2^{\text{exposant}}$

$s = 0$ (si le nombre est positif) ou $s = 1$ (si le nombre est négatif)

La liste des bits désignés par $xxx...xxx$ s'appelle la mantisse.

L'exposant est un entier relatif.

Exemple2 :

$$(-1)^0 \times (1,001001011)_2 \times 2^5 = 1 \times (1,001001011)_2 \times 2^5 = (100100,1011)_2 = 36,1875$$

$$(-1)^1 (1,001)_2 \times 2^{-2} = (-1) \times (0,01001)_2 = -(0,01001)_2 = -0,28125$$

$$-3,25 = -(101,01)_2 = (-1)^1 \times (1,0101)_2 \times 2^2$$

Exercice :

☛ donner l'écriture en virgule flottante des réels suivants écrits en décimal :

• $17,625 = 10001,101 = (-1)^0 \times (1,001101)_2 \times 2^4$

• $0,10,000110011... = (-1)^0 \times (1,10011...)_2 \times 2^{-1}$

• $\frac{1}{3}$

☛ Écrire en décimal le nombre qui a pour écriture en virgule flottante $(-1)^1 \times (1,01101)_2 \times 2^2$

Principe de codage de cette écriture en machine sur n bits :

On réserve un bit pour le signe, un certain nombre de bits pour l'exposant, et les autres pour coder la mantisse.

Le chiffre avant la virgule étant forcément 1 on ne le représente pas en machine.

Selon les applications ou les machines on peut avoir différentes règles pour représenter en machine les écritures en virgule flottante : choix du nombre de bits pour la mantisse, pour l'exposant, etc.

La norme internationale IEEE 754 définit des règles communes de représentation des flottants selon 2 formats :

- Le format simple précision : 32 bits sont utilisés pour représenter le nombre.
- Le format double précision : 64 bits sont utilisés pour représenter le nombre.

Remarque : Python utilise le format double précision.

Tableau récapitulatif :

Précision	Encodage	Signe	Exposant	Mantisse	Valeur d'un nombre	Chiffres significatifs
Simple	32 bits	1 bit	8 bits	23 bits	$(-1)^s \times M \times 2^{E-127}$	environ 7
Double	64 bits	1 bit	11 bits	52 bits	$(-1)^s \times M \times 2^{E-1023}$	environ 16

Remarque : le codage de l'exposant est décalé afin de le stocker sous forme d'un entier non signé.

Exemple : codage de 9,8125

- $9,8125 = (1001,1101)_2 = 1,0011101 \times 2^3$
- La mantisse est alors 0011101.
- La mantisse étant constituée de 23 bits, il faut rajouter des 0 à droite des bits : 00111010000000000000000.
- Coder 3 revient à coder 130 donc l'exposant est 10000010.
- On accole tous les morceaux, le signe + est codé par 0.
On obtient donc 01000001000111010000000000000000.

À vous de jouer :

prouver que la représentation de $-21,375$ en suivant la norme IEE-754 est 10000011010101100000000000000000 soit en hexadécimal 0xc1ab0000.

$-21,375 = (-10101,011)_2 = 1,0101011 \times 2^4$
Mantisse : 0101011... sur 23 bits : 01010110000000000000000
 $4 + 127 = 131_{10} = 128 + 2 + 1 = 10000011_2$ Signe négatif $\rightarrow 1$
11000001101010110000000000000000

II. EXERCICES SUPPLÉMENTAIRES

1. Coder en binaire $\frac{81}{16}, \frac{17}{32}, 16, 25$
2. Le 25 février 1991, pendant la Guerre du Golfe, des missiles Patriot, pilotés informatiquement, ont échoué dans l'interception d'un missile Scud irakien qui a tué 28 soldats.
La commission d'enquête a conclu à un calcul incorrect du temps de parcours en machine.
Sachant que le temps était compté en dixième de seconde pouvez donner une explication à cet échec.
3. On donne le programme ci-contre :
Que se passe-t-il ?

```
1 x=1
2 while x!=0:
3     x=x-1
```
4. Soit le nombre flottant au format simple précision : 00111101110011001100110011001100.
Trouvez la représentation en base 10 de ce nombre.
5. Déterminez la représentation au format simple précision de 4,375 en binaire et en hexadécimal.
6. Déterminez la conversion décimale de 01000100111111001000010000000000, un nombre écrit avec la norme IEEE754 en simple précision.
7. Déterminez la représentation au format simple précision de 0,2 en binaire.

III. VIDÉOS ET SITES

Nombre décimal en binaire

<https://www.youtube.com/watch?v=Tmg8BgIGPes>



Norme IEEE754

<https://www.youtube.com/watch?v=VYvFso8DYnU>



Pour vérifier vos résultats écrits à l'aide de la norme IEEE-754 :

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

